



SMART CONTRACT SECURITY ASSESSMENT

OffYield

— **YIELD VAULT**

SECURITY REVIEW

11 SEPTEMBER 2026

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

Contents

1	About Shieldify	3
2	Disclaimer	3
3	About OffYield	3
4	Risk Classification	4
4.1	Impact	4
4.2	Likelihood	4
5	Security Review Summary	4
5.1	Protocol Summary	5
5.2	Scope	5
6	Findings Summary	5
7	Findings	6

1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3 About OffYield

OffYield is a non-custodial yield-spending vault. A user deposits USDG into `OffyieldVault`, which supplies the assets to an external ERC-4626 yield venue and records the depositor's face-value principal and venue-share balance in a per-user position. Only the interest accrued above that recorded principal becomes spendable, so the deposit itself is never consumed by spending activity.

The intended deployment routes deposits into a Morpho Vault V2 instance (the Steakhouse USDG vault), which means the downstream ERC-4626 quote - rather than an internal price - is the source of truth for position value, impairment, and spendable yield.

At a high level, the main functionalities are:

- Users deposit USDG and receive a position tracking `principal`, venue `shares`, and cumulative `spent`
- Value accrued above the share reservation covering principal is exposed as spendable yield and drawn through `spendFromYield()` to a caller-selected receiver
- `withdrawPrincipal()` supports both a healthy branch and an impaired branch that debits principal at face value and pays out pro rata

- `withdrawPrincipal()` and `exit()` are unpausable withdrawal routes; deposits and yield spending are pausable by a guardian
- The owner configures deposit limits (`maxSingleDeposit` , `tvlCap`) and rotates the guardian role

4 Risk Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	 CRITICAL	 HIGH	 MEDIUM
Likelihood: Medium	 HIGH	 MEDIUM	 LOW
Likelihood: Low	 MEDIUM	 LOW	 LOW

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable, and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5 Security Review Summary

The security review lasted 4 days with a total of 64 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written, with a clearly stated invariant set and an explicit separation between principal and spendable yield. The audit report contributed by identifying two Medium, four Low severity issues. They're mainly related to yield accounting, liquidity and exit assumptions, deposit/share accounting, slippage protection, configuration validation, and internal ledger consistency.

The OffYield team has done a great job with their invariant documentation and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

PROJECT NAME	OFFYIELD
Repository	offyield-contracts
Type of Project	Yield-Bearing Stablecoin Vault (Spendable-Yield Card Product)
Security Review Timeline	4 days
Review Commit Hash	60c7af0f3afe545e395d7c15036a568e429352a9
Fixes Review Commit Hash	12183f59626fb32b3a043624a73f0c5f1ec403a9

5.2 Scope

The following smart contracts were in the scope of the audit:

FILE	NSLOC
src/OffyieldVault.sol	219
Total	219

6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: **2**
- **Low** issues: **4**
- **Info** issues: **6**

ID	TITLE	SEVERITY	STATUS
[M-01]	Provisional Morpho Vault V2 Interest Can Be Exposed as Spendable Yield	MEDIUM	FIXED
[M-02]	Unpausable Exits Rely on Untimelocked Liquidity, Leaving OffyieldVault No Self-Help	MEDIUM	FIXED
[L-01]	Deposit Does Not Verify That Reported Venue Shares Were Actually Minted	LOW	FIXED
[L-02]	Principal Withdrawals and Exits Lack Slippage Bounds	LOW	FIXED

ID	TITLE	SEVERITY	STATUS
[L-03]	<code>setCaps()</code> Accepts Nonsensical and Instantly Freezing Configurations	LOW	FIXED
[L-04]	<code>exit()</code> and <code>spendFromYield()</code> Disagree on the <code>p.spent</code> Ledger	LOW	FIXED
[I-01]	A Reverting Morpho Market Can Block OffYield Value Operations	INFO	FIXED
[I-02]	Yield-Only Positions Have No Partial Unpausable Exit During Venue Illiquidity	INFO	FIXED
[I-03]	Documentation Contradicts Itself About View Staleness During Unrealized Venue Losses	INFO	FIXED
[I-04]	<code>tvlCap</code> Is Enforced Against Face-Value Principal the Vault No Longer Backs	INFO	FIXED
[I-05]	The Documented "One Share" Tolerance Is Implemented as One Share-Wei	INFO	FIXED
[I-06]	A 2-of-2 Owner Multisig Has No Fault Tolerance	INFO	FIXED

7 Findings

M-01 MEDIUM FIXED Provisional Morpho Vault V2 Interest Can Be Exposed as Spendable Yield

Description

`OffyieldVault` treats the downstream ERC-4626 vault quote as the source of truth for position value and spendable yield. This is correct for OffYield's internal share accounting, but it is a weaker guarantee than "only definitively realized yield is spendable."

For the intended Steakhouse USDG integration, the downstream quote depends on Morpho Vault V2 accounting. Morpho documents a phantom-interest state where an allocated market can continue reporting accrued borrower interest even though that interest may later prove unrecoverable because collateral is illiquid or insolvent, or liquidations are broken or uneconomical. During that window, the Morpho Vault V2 share price can be higher than the value that will ultimately be recoverable.

OffYield has no independent recoverability check. If the venue share price includes provisional interest, `spendableYieldOf()` exposes the value above the principal share reservation as spendable. A user can then call `spendFromYield()` and receive real USDG liquidity before the venue later realizes the corresponding loss.

Location of Affected Code

File: [src/OffyieldVault.sol](#)

```
function positionValue(address user) public view returns (uint256) {
    return yieldVault.convertToAssets(_positions[user].shares);
}

function spendableYieldOf(address user) public view returns (uint256) {
    return yieldVault.convertToAssets(_spendableShares(_positions[user]));
}

function spendFromYield(uint256 assets, address receiver) external nonReentrant whenNotPaused {
    if (assets == 0) revert ZeroAmount();
    if (receiver == address(0)) revert ZeroAddress();

    Position storage p = _positions[msg.sender];
    uint256 spendableShares = _spendableShares(p);
    uint256 sharesNeeded = yieldVault.previewWithdraw(assets);
    if (sharesNeeded > spendableShares) {
        revert ExceedsSpendableYield(assets, yieldVault.convertToAssets(spendableShares));
    }

    p.shares -= sharesNeeded;
    p.spent += assets;
    totalShares -= sharesNeeded;

    uint256 sharesBurned = _venueWithdraw(assets, receiver);
    if (sharesBurned > sharesNeeded) revert PrincipalBreach();
    uint256 refund = sharesNeeded - sharesBurned;
    p.shares += refund;
    totalShares += refund;

    emit YieldSpent(msg.sender, receiver, assets, sharesBurned);
}

function _spendableShares(Position storage p) internal view returns (uint256) {
    if (p.principal == 0) return p.shares;
    uint256 sharesForPrincipal = yieldVault.previewWithdraw(p.principal);
    return p.shares > sharesForPrincipal ? p.shares - sharesForPrincipal : 0;
}
```

Impact

A user who acts during a Morpho phantom-interest window can spend yield that is only provisionally reflected in the downstream share price. If the venue later realizes bad debt, the already-spent USDG has left the venue, while the subsequent lower share price is borne by the remaining venue shares.

Proof of Concept

1. Alice deposits `10,000e6` USDG.
2. Bob deposits `10,000e6` USDG.
3. The venue share price rises because it reports `20,000e6` of apparent interest.
4. `spendableYieldOf(alice)` returns nearly `10,000e6`, because Alice's shares now quote near `20,000e6` while her principal reservation remains near `10,000e6`.
5. Alice calls `spendFromYield()` for the quoted spendable amount and receives real USDG.
6. The venue later realizes a `20,000e6` loss.
7. Alice's remaining shares and Bob's shares are both repriced downward, but Alice has already withdrawn liquidity through the spend path.

Recommendation

Do not treat raw Morpho Vault V2 `convertToAssets()` appreciation as immediately spendable OffYield yield for this integration.

Team Response

Fixed.

M-02

MEDIUM

FIXED

Both Unpausable Exits Depend on the Venue's Untimelocked Liquidity Route, and `OffyieldVault` Has No Self-Help Lever

Description

`exit()` and `withdrawPrincipal()` are `OffyieldVault`'s two unpausable exit paths, and both settle through the venue: `exit()` calls `yieldVault.redeem()`, `withdrawPrincipal()` calls `yieldVault.withdraw()`. Inside `VaultV2` both land in the same internal `exit()`, which covers any shortfall between the requested assets and the venue's idle balance by deallocating through `liquidityAdapter` – and only when `liquidityAdapter != address(0)`.

`liquidityAdapter` is written by `setLiquidityAdapterAndData()`, which is allocator-only and carries no timelock. Setting it to `address[0]` while the venue holds no idle assets removes `VaultV2`'s only route to raise withdrawal liquidity, so both of `OffyieldVault`'s unpausable exits revert for every user. This contradicts the contract header's claim that principal withdrawal and exit "can never be paused, by anyone, in any state."

The in-scope defect is not that the venue can run dry – it is that `OffyieldVault` cannot pull its own liquidity back, while a direct venue LP can. `VaultV2`'s remedy is `forceDeallocate()`: permissionless, callable by any holder of venue shares, and paid for with a share-denominated penalty. `OffyieldVault` never calls it and approves no one to spend its venue shares, so once `forceDeallocatePenalty` is nonzero the wrapper cannot be rescued at all – a third party calling `forceDeallocate(..., onBehalf = OffyieldVault)` reverts when `VaultV2` tries to take the penalty against a zero allowance.

Location of Affected Code

Both unpausable exits reach the venue, and neither has a fallback:

File: [src/OffyieldVault.sol](#)

```
// exit (line 336) — the unpausable full exit
uint256 paid = yieldVault.redeem(shares, receiver, address(this));

// _venueWithdraw (line 393), reached by the unpausable withdrawPrincipal
sharesBurned = yieldVault.withdraw(assets, receiver, address(this));
```

The yield venue sources withdrawal liquidity only through `liquidityAdapter` (lines 800–807):

File: `VaultV2.sol` (yield venue)

```
function exit(uint256 assets, uint256 shares, address receiver, address onBehalf) internal {
    // code
    uint256 idleAssets = IERC20(asset).balanceOf(address(this));
    if (assets > idleAssets && liquidityAdapter != address[0]) {
        deallocateInternal(liquidityAdapter, liquidityData, assets - idleAssets);
    }
    // code
}
```

Nulling that route is allocator-only with no timelock:

File: `VaultV2.sol` (yield venue)

```
function setLiquidityAdapterAndData(address newLiquidityAdapter, bytes memory newLiquidityData) external {
    require(isAllocator[msg.sender], ErrorsLib.Unauthorized());
    liquidityAdapter = newLiquidityAdapter;
    liquidityData = newLiquidityData;
```

```
// code
}
```

The remedy `OffyieldVault` cannot reach – the penalty is withdrawn from `onBehalf`, which requires either being the caller or having granted it an allowance:

File: `VaultV2.sol` (yield venue)

```
function forceDeallocate(address adapter, bytes memory data, uint256 assets, address onBehalf) external returns
(uint256) {
    bytes32[] memory ids = deallocateInternal(adapter, data, assets);
    uint256 penaltyAssets = assets.mulDivUp(forceDeallocatePenalty[adapter], WAD);
    uint256 penaltyShares = withdraw(penaltyAssets, address(this), onBehalf);
    // code
}
```

Impact

Both exit paths revert for every `OffyieldVault` user until an unrelated party intervenes, triggered by a single untimelocked allocator call. `OffyieldVault` has no way to restore its own liquidity, so its depositors are strictly worse off than they would be holding the venue shares directly.

Proof of Concept

Initial state:

- Alice holds a 10,000 USDG position in `OffyieldVault`, which wraps the live `VaultV2`.
- `forceDeallocatePenalty[ADAPTER]` is 0, its live value at time of testing.

Steps:

1. The curator acts maliciously and grants an address allocator status (no timelock), and that address calls `setLiquidityAdapterAndData(address[0], "")`.
2. With no liquidity route and no idle assets, `exit(alice)` reverts, and `withdrawPrincipal(1,000 USDG, alice)` reverts too.
3. While the penalty is 0, the block is trivially cleared: any address – even one holding no venue shares – calls `forceDeallocate(ADAPTER, liquidityData, 20,000 USDG, onBehalf = itself)` for free, and Alice's `exit()` then succeeds.
4. The curator raises `forceDeallocatePenalty[ADAPTER]` to 20 bps (`0.0020e18`). The free rescue is now gone: a caller with no venue shares cannot pay the penalty, so the call reverts.
5. Calling `forceDeallocate(..., onBehalf = address(OffyieldVault))` reverts as well. `OffyieldVault` holds venue shares but has approved no spender, so `VaultV2`'s penalty withdrawal fails against a zero allowance.

Recommendation

Add a permissioned passthrough on `OffyieldVault` that calls `yieldVault.forceDeallocate(adapter, data, assets, address[this])` so that it can pull its own funds back out of the venue's markets.

Team Response

Fixed.

L-01

● LOW

FIXED

Deposit Does Not Verify That Reported Venue Shares Were Actually Minted

Description

`deposit()` trusts the `shares` value returned by `yieldVault.deposit(assets, address[this])` and records it directly into the user's position and `totalShares`. The function verifies the inbound asset transfer amount, but it does not snapshot the vault's venue-share balance before and after the deposit to confirm that the returned shares were actually minted to `OffyieldVault`.

In any case where an ERC-4626 venue returns more shares than it minted, `OffyieldVault` records phantom shares. Because `credited` is computed from the returned `shares` and then capped at `assets`, the over-report can pass the existing `credited <= assets` and one-share tolerance checks. The inflated share ledger can then create fabricated spendable yield and can later consume real shares deposited by other users.

Location of Affected Code

File: [src/OffyieldVault.sol](#)

```
function deposit(uint256 assets) external nonReentrant whenNotPaused {
    // code
    asset.forceApprove(address(yieldVault), assets);
    uint256 shares = yieldVault.deposit(assets, address(this));

    uint256 credited = yieldVault.convertToAssets(shares);
    if (credited > assets) credited = assets;
    if (shares == 0 || credited == 0) revert ZeroAmount();
    if (credited + _oneShareValue() < assets) revert UnexpectedTransferAmount();

    Position storage p = _positions[msg.sender];
    p.principal += credited;
    p.shares += shares;
    totalPrincipal += credited;
    totalShares += shares;
```

```
//code
```

```
}
```

Impact

An over-reporting venue can make a depositor appear to own shares the vault does not actually hold. The user can spend fabricated yield, and any remaining phantom shares can later be redeemed against real venue shares supplied by later depositors. Remaining users can become under-backed even though their own position ledgers are unchanged.

Proof of Concept

1. Alice deposits `10,000e6` assets into `OffyieldVault`.
2. The venue mints the honest `10,000e6` shares but returns `20,000e6` from `deposit()`.
3. `OffyieldVault` records Alice with `20,000e6` shares and `10,000e6` principal.
4. Alice immediately calls `spendFromYield(10,000e6, receiver)`.
5. The venue burns Alice's only real `10,000e6` shares and pays `10,000e6`; Alice still has `10,000e6` recorded phantom shares and `10,000e6` principal.
6. Bob later deposits `10,000e6` through an honest deposit path.
7. Alice calls `exit(receiver)` and redeems her remaining phantom shares against Bob's real backing.

Recommendation

Snapshot the venue share balance before and after `yieldVault.deposit()` and require the actual balance delta to equal the returned `shares`.

```
uint256 shareBalBefore = yieldVault.balanceOf(address(this));
uint256 shares = yieldVault.deposit(assets, address(this));
if (yieldVault.balanceOf(address(this)) - shareBalBefore != shares) {
    revert PrincipalBreachd();
}
```

Team Response

Fixed.

Principal Withdrawals and Exits Lack Slippage Bounds

Description

`withdrawPrincipal()` and `exit()` use the downstream ERC-4626 venue's value at execution time, but accept no caller-selected `minAssetsOut` or deadline.

If the venue loses value after a user submits a transaction, the transaction can still succeed at the lower execution-time price. In `withdrawPrincipal()` this affects the impaired branch, which only rejects a zero payout. `exit()` accepts whatever `redeem()` returns, including zero where the venue permits zero-asset redemption.

Location of Affected Code

File: [src/OffyieldVault.sol#L238-L331](#)

```
function withdrawPrincipal(uint256 assets, address receiver) external nonReentrant {
    // code
    uint256 value = yieldVault.convertToAssets(p.shares);
    if (value >= p.principal) {
        // Healthy branch: requests the exact asset amount or reverts.
        p.principal -= assets;
        totalPrincipal -= assets;
        uint256 sharesBurned = _venueWithdraw(assets, receiver);
        p.shares -= sharesBurned;
        totalShares -= sharesBurned;
    } else {
        uint256 sharesToBurn = p.shares * assets / p.principal;
        if (sharesToBurn == 0) revert ZeroAmount();
        p.principal -= assets;
        totalPrincipal -= assets;
        p.shares -= sharesToBurn;
        totalShares -= sharesToBurn;
        uint256 paid = _venueRedeem(sharesToBurn, receiver);
        if (paid == 0) revert ZeroAmount();
    }
}

function exit(address receiver) external nonReentrant {
    // code
    p.shares = 0;
    p.principal = 0;
    totalShares -= shares;
    totalPrincipal -= principalDebited;
    uint256 paid = yieldVault.redeem(shares, receiver, address(this));
```

```
emit PositionExited(msg.sender, receiver, paid, principalDebited, shares);  
}
```

Impact

An impaired partial withdrawal or full exit can settle materially below the value observed before submission. `exit()` clears the position after redemption, so a later venue recovery cannot restore the user's claim. In an extreme zero-asset state, it may clear the position without transferring assets.

The impact requires a venue loss or accounting/configuration change before execution and a venue that still permits redemption. This is a user-protection issue, not cross-user theft.

Proof of Concept

1. Seed the venue with `1,000,000e6` and deposit `10,000e6` for Alice.
2. Alice observes a value near `10,000e6` and prepares `exit(alice)`.
3. Reduce venue assets to `100e6` before execution.
4. `exit(alice)` succeeds, pays `990,099` units, and clears Alice's principal and shares.

Recommendation

Add caller-selected bounds:

```
function withdrawPrincipal(  
    uint256 assets,  
    address receiver,  
    uint256 minAssetsOut,  
    uint256 deadline  
) external;  
  
function exit(  
    address receiver,  
    uint256 minAssetsOut,  
    uint256 deadline  
) external;
```

Team Response

Fixed.

L-03

LOW

FIXED

setCaps() Accepts Nonsensical and Instantly Freezing Configurations

Description

`setCaps()` writes both `maxSingleDeposit` and `tvICap` straight to storage with no validation against the vault's current state or against each other.

Two configurations pass that no operator would intend. Setting `tvICap` below the current `totalPrincipal` freezes deposits product-wide, existing users included: every later deposit finds `totalPrincipal` already over the cap and reverts with `CapExceeded`, and that revert is the only signal anything changed. Setting `maxSingleDeposit` above `tvICap` is accepted too and is simply dead configuration, since no single deposit can approach it without breaching `tvICap` first. The call is single-step and non-timelocked, so either can also land in front of deposits already in flight.

Location of Affected Code

Both caps are written with no checks:

File: [src/OffyieldVault.sol#L238-L331](#)

```
function setCaps(uint256 maxSingleDeposit_, uint256 tvICap_) external onlyOwner {
    maxSingleDeposit = maxSingleDeposit_;
    tvICap = tvICap_;
    emit CapsSet(maxSingleDeposit_, tvICap_);
}
```

Impact

A single owner call can silently freeze every new deposit, for existing and new users alike, with a call-time `CapExceeded` as the only indication. The same call can leave `maxSingleDeposit` set to a value that never binds.

Proof of Concept

Initial state:

- Alice has deposited 1,000 USDG, so `totalPrincipal` is 1,000 USDG.

Steps:

1. The owner calls `setCaps(500 USDG, 1 USDG)`. The new `tvICap` of 1 USDG is far below `totalPrincipal`, and the call succeeds with no revert.
2. Bob's next deposit of 1 USDG reverts with `CapExceeded`, as does every deposit after it – the product is frozen for new deposits with no prior signal.
3. In a separate call, the owner sets `setCaps(10,000 USDG, 100 USDG)`. This is accepted as well.
4. That `maxSingleDeposit` of 10,000 USDG can never bind, since any deposit approaching it breaches the 100 USDG `tvICap` first.

Recommendation

Validate both parameters before writing them, and emit the previous values alongside the new ones so a misconfiguration is visible in the event log:

```
require(tvICap_ == 0 || tvICap_ >= totalPrincipal, "tvICap below totalPrincipal");
require(tvICap_ == 0 || maxSingleDeposit_ <= tvICap_, "maxSingleDeposit exceeds tvICap");
```

Team Response

Fixed.

L-04

● LOW

FIXED

`exit()` and `spendFromYield()` Disagree on the `p.spent` Ledger

Description

A position can be emptied of all its value in two different ways, and the two paths do not maintain the same storage.

`p.spent` has exactly one writer – `spendFromYield()`, which does `p.spent += assets`. Nothing ever decrements it, resets it, or deletes the position slot.

Path A, `exit()`. It redeems the position's entire share balance, paying principal *plus every unit of accrued yield*, then zeroes `p.shares` and `p.principal`. It never touches `p.spent`. Yield leaves the vault and the spend ledger does not move, so `spentOf()` **under-reports** by the whole yield component of every exit.

Path B, `spendFromYield()` on a principal-free position. `_spendableShares()` short-circuits to the position's *entire* balance when `p.principal == 0`:

```
if (p.principal == 0) return p.shares;
```

`p.principal` reaches 0 while shares remain whenever a user withdraws their full principal and leaves the yield behind, which `withdrawPrincipal()` permits in a single call. From there `spendFromYield()` can spend the position down until it holds no value at all. The position is now empty by every economic measure – `principalOf == 0`, `positionValue == 0` – but `p.spent` keeps the accumulated figure and is never cleared, because the only code that empties a position this way is the same code that increments the counter.

The result is that two positions with identical deposits, identical yield, and identical terminal state report contradictory `spentOf()` values: 0 for the one closed by `exit()`, the full yield for the one closed by spending.

Location of Affected Code

The single writer, in `spendFromYield()`:

File: [src/OffyieldVault.sol#L238-L331](#)

```
function spendFromYield(uint256 assets, address receiver) external nonReentrant whenNotPaused {  
    // code  
    p.shares -= sharesNeeded;  
    p.spent += assets;    // the only write to `spent`, anywhere  
    totalShares -= sharesNeeded;  
    // code  
}
```

Path A – `exit()` zeroes the position and pays out yield without crediting `spent`, and without deleting the slot:

File: [src/OffyieldVault.sol#L316-L332](#)

```
function exit(address receiver) external nonReentrant {  
    // code  
    p.shares = 0;  
    p.principal = 0;  
    // code  
    uint256 paid = yieldVault.redeem(shares, receiver, address(this)); // principal + ALL unspent yield  
    emit PositionExited(msg.sender, receiver, paid, principalDebited, shares);  
}  
// no `p.spent` write, no `delete _positions[msg.sender]`
```

Path B – with principal at zero, the whole balance becomes spendable:

File: [src/OffyieldVault.sol#L400-L404](#)

```
function _spendableShares(Position storage p) internal view returns (uint256) {
    if (p.principal == 0) return p.shares; // spend can empty the position outright
    uint256 sharesForPrincipal = yieldVault.previewWithdraw(p.principal);
    return p.shares > sharesForPrincipal ? p.shares - sharesForPrincipal : 0;
}
```

Impact

Two users with identical histories read different `spentOf()` values depending only on which exit route they took.

Recommendation

Pick one meaning for `spent` and make **both** paths honour it. For example, credit the yield component in the `exit()` path:

```
// in exit(), after the redeem
uint256 paid = yieldVault.redeem(shares, receiver, address(this));
p.spent += paid > principalDebited ? paid - principalDebited : 0;
```

Team Response

Fixed.

I-01

● INFO

FIXED

A Reverting Morpho Market Can Block OffYield Value Operations

Description

`OffyieldVault` relies on Morpho Vault V2 for valuation and liquidity. Its Morpho adapter loops through all active markets in `realAssets()` and does not isolate a reverting `expectedSupplyAssets()` call. The revert then prevents Vault V2 interest accrual, conversions, previews, deposits, withdrawals, and redemptions from completing.

Location of Affected Code

File: [src/OffyieldVault.sol](#)

```
function positionValue(address user) public view returns (uint256) {
    return yieldVault.convertToAssets(_positions[user].shares);
}

function spendableYieldOf(address user) public view returns (uint256) {
    return yieldVault.convertToAssets(_spendableShares(_positions[user]));
}
```

```

}

function deposit(uint256 assets) external nonReentrant whenNotPaused {
    // Calls yieldVault.convertToAssets() through _materiallyImpaired().
    if (_materiallyImpaired(_positions[msg.sender])) revert DepositWhileImpaired();
    // The venue's deposit() also calls Vault V2 accrueInterest().
    uint256 shares = yieldVault.deposit(assets, address(this));
    uint256 credited = yieldVault.convertToAssets(shares);
    // ...records principal and shares after the venue calls succeed.
}

function spendFromYield(uint256 assets, address receiver)
    external nonReentrant whenNotPaused
{
    uint256 spendableShares = _spendableShares[_positions[msg.sender]];
    uint256 sharesNeeded = yieldVault.previewWithdraw(assets);
    // ...calls yieldVault.withdraw() after the preview succeeds.
}

function withdrawPrincipal(uint256 assets, address receiver) external nonReentrant {
    Position storage p = _positions[msg.sender];
    uint256 value = yieldVault.convertToAssets(p.shares);
    // ...chooses a healthy or impaired branch only after this call succeeds.
}

function exit(address receiver) external nonReentrant {
    Position storage p = _positions[msg.sender];
    uint256 shares = p.shares;
    uint256 principalDebited = p.principal;
    p.shares = 0;
    p.principal = 0;
    totalShares -= shares;
    totalPrincipal -= principalDebited;
    uint256 paid = yieldVault.redeem(shares, receiver, address(this));
}

```

The corresponding upstream paths are `MorphoMarketV1AdapterV2.realAssets()` and Vault V2's `accrueInterestView()`, `convertToAssets()`, `previews`, `deposit()`, `withdraw()`, and `redeem()`:

- <https://github.com/morpho-org/vault-v2/blob/main/src/adapters/MorphoMarketV1AdapterV2.sol#L17-L20>
- <https://github.com/morpho-org/vault-v2/blob/main/src/adapters/MorphoMarketV1AdapterV2.sol#L223-L253>
- <https://github.com/morpho-org/vault-v2/blob/main/src/VaultV2.sol#L618-L679>
- <https://github.com/morpho-org/vault-v2/blob/main/src/VaultV2.sol#L702-L740>

Impact

- `positionValue()`, `spendableYieldOf()`, `isImpaired()`, and `impairmentOf()` can revert.
- New deposits can revert during the impairment check or the venue deposit.
- `spendFromYield()` can revert during venue previews or withdrawal.
- `withdrawPrincipal()` can revert before selecting its healthy or impaired branch.
- `exit()` can revert when Vault V2's `redeem()` attempts to accrue interest.

Proof of Concept

1. Alice deposits `10,000e6` USDG and receives principal and venue shares.
2. An active market with nonzero adapter shares makes `expectedSupplyAssets()` revert.
3. The adapter's `realAssets()` and Vault V2's `accrueInterestView()` revert.
4. OffYield's value views, `withdrawPrincipal()`, and `exit()` revert. Failed state-changing calls roll back, so Alice's position remains recorded but inaccessible.

Recommendation

Add an unpausable in-kind exit that transfers the user's recorded venue shares without calling valuation, previews, `withdraw()`, or `redeem()`. Update the ledger only after a successful transfer and validate Vault V2 share gates and balance deltas.

Team Response

Fixed.

I-02

● INFO

FIXED

Yield-Only Positions Have No Partial Unpausable Exit During Venue Illiquidity

Description

`exit()` is the only unpausable route that can withdraw a position after its principal has reached zero, but it is all-or-nothing. It always attempts to redeem every remaining share in one call.

`withdrawPrincipal()` cannot help once `p.principal == 0`, because any positive request fails the `assets > p.principal` check. The only partial route for a yield-only position is `spendFromYield()`, which is protected by `whenNotPaused`.

Consequently, a guardian pause combined with partial venue illiquidity can wedge a yield-only position even when the venue can pay part of its value. A full `exit()` requests the entire position and reverts if the venue cannot satisfy that complete redemption.

Location of Affected Code

File: [src/OffyieldVault.sol#L201-L243](#)

File: [src/OffyieldVault.sol#L316-L330](#)

```
function spendFromYield(uint256 assets, address receiver) external nonReentrant whenNotPaused {  
    // The only partial value-withdrawal path for a yield-only position.  
}  
  
function withdrawPrincipal(uint256 assets, address receiver) external nonReentrant {  
    // code  
    if (assets > p.principal) revert ExceedsPrincipal(assets, p.principal);  
    // A yield-only position has p.principal == 0, so every positive request reverts.  
}  
  
function exit(address receiver) external nonReentrant {  
    // code  
    uint256 shares = p.shares;  
    // The entire remaining position is redeemed in one call.  
    uint256 paid = yieldVault.redeem(shares, receiver, address(this));  
}
```

Impact

A user who has already withdrawn all principal but left accrued yield in the venue cannot withdraw any amount while the vault is paused if the venue cannot serve the entire remaining position.

The issue does not take value from another user and does not create an accounting loss. It is a liveness failure for the yield-only cohort and contradicts the documented claim that pause or owner-key loss cannot trap any user value at this layer.

The impact requires both an active pause and venue liquidity that is positive but below the user's full remaining position. If the venue can redeem the full position, `exit()` remains available.

Proof of Concept

Using the supplied mock venue:

1. Alice deposits `10,000e6` and the venue accrues enough yield to make her position worth approximately `19,801e6`.
2. Alice calls `withdrawPrincipal(10,000e6, alice)`, leaving `principal == 0` and shares worth approximately `19,801e6`.
3. The guardian calls `pause()`.
4. Reduce venue liquidity so it can serve `9,900e6` but not the full `19,801e6` position.
5. `withdrawPrincipal(1, alice)` reverts with `ExceedsPrincipal`.

6. `spendFromYield(...)` reverts with `EnforcedPause`.
7. `exit(alice)` requests the full position and reverts at the venue because the full amount is unavailable.

Thus, the position has redeemable partial liquidity but no successful OffYield entry point.

Recommendation

Add a partial, unpausable share-based exit for the yield-only bucket:

```
function withdrawShares(uint256 shares, address receiver) external nonReentrant {  
    // Require shares <= _spendableShares[_positions[msg.sender]] when principal > 0.  
    // For principal == 0, allow shares <= p.shares.  
    // Debit only the requested shares and redeem them from the venue.  
}
```

Alternatively, parameterize `exit()` with a share amount and zero `principal` only when the final shares are redeemed. The partial route must remain bounded so it cannot consume shares reserved for principal.

Team Response

Fixed.

I-03

● INFO

FIXED

Documentation Contradicts Itself About View Staleness During Unrealized Venue Losses

Description

The documentation makes two incompatible claims about the direction of stale venue views.

`README.md` venue fact #4 states that a pending venue loss is invisible until an interaction realizes it. Under that model, `positionValue()`, `isImpaired()`, `impairmentOf()`, and `spendableYieldOf()` can temporarily read too high.

`audit/INVARIANTS.md` non-invariant #4 instead says views can read low, never high, and describes stale accrued interest as the only stale direction. Both statements cannot hold for the same venue behavior.

The contract exposes the venue's view directly through the following paths:

```
function positionValue(address user) public view returns (uint256) {  
    return yieldVault.convertToAssets[_positions[user].shares];  
}
```

```
function isImpaired(address user) public view returns (bool) {  
    Position storage p = _positions[user];  
    return yieldVault.convertToAssets(p.shares) < p.principal;  
}
```

Location of Affected Code

File: [src/OffyieldVault.sol#L112-L150](#)

File: [README.md#L54-L61](#)

File: [audit/INVARIANTS.md#L63-L71](#)

Impact

Monitoring, user interfaces, and automated payment flows may interpret an inconsistent view as authoritative. In particular, a position can appear to have spendable yield while the corresponding spend fails after the venue realizes a loss.

Recommendation

Reconcile the README and invariant specification with the actual deployed venue behavior. State explicitly whether views:

- include live adapter balances;
- can be stale during pending gains, losses, or both; and
- are suitable for user-facing spendable-yield displays.

Team Response

Fixed.

I-04

● INFO

FIXED

tvICap Is Enforced Against Face-Value
Principal the Vault No Longer Backs

Description

`deposit()` measures `tvICap` against `totalPrincipal`, which is a face-value figure: it records what LPs contributed, not what the vault currently holds. Impaired positions keep principal at face value by design, and the impaired branch of `withdrawPrincipal()` debits it at face value too, so a venue loss never reduces `totalPrincipal`.

After a loss, `totalPrincipal` therefore overstates real backing by the size of the loss, and deposit headroom shrinks against principal that nothing backs – exactly when the protocol would most want fresh capital. This is a capacity-accounting defect, no funds are at risk, and it self-heals as impaired positions exit.

Location of Affected Code

The cap is measured against `totalPrincipal`:

File: [src/OffyieldVault.sol#L159](#)

```
function deposit(uint256 assets) external nonReentrant whenNotPaused {  
    // code  
    if (tvlCap != 0 && totalPrincipal + assets > tvlCap) revert CapExceeded();  
    // code  
}
```

The impaired branch debits that same figure at face value, so a loss never shrinks it:

File: [src/OffyieldVault.sol#L248-L249](#)

```
function withdrawPrincipal(uint256 assets, address receiver) external nonReentrant {  
    // code  
} else {  
    // Impaired: debit principal at face value, pay out the pro-rata  
    // share of what actually remains.  
    // code  
    p.principal -= assets;  
    totalPrincipal -= assets; // face value, not the reduced real value  
    // code  
}  
//code  
}
```

Impact

Deposit capacity is reduced by the full size of any venue loss, even though the cap is meant to bound the vault's real size rather than its historical inflows. Depositors who would leave the vault well inside its intended limit are turned away until impaired positions unwind.

Proof of Concept

Initial state:

- `tvlCap` is 2,000 USDG. Alice has deposited 1,000 USDG and Bob 500 USDG, so `totalPrincipal` is 1,500 USDG, fully backed.

Steps:

1. A venue loss wipes out 50% of the venue's balance. `totalPrincipal` stays at 1,500 USDG; real backing (`convertToAssets(totalShares)`) falls to roughly 750 USDG.
2. Charlie deposits 501 USDG. The check computes `1500 + 501 > 2000` and reverts with `CapExceeded`.
3. Real headroom at that moment is about 1,250 USDG – a 2,000 USDG cap against roughly 750 USDG actually held. The cap instead offers 500 USDG, measured against 750 USDG of principal that nothing backs.

Final state: a deposit comfortably inside the vault's true capacity is rejected.

Recommendation

Either document `tvlCap` as a cumulative inflow limit rather than a current-value cap, or measure it against real backing – `yieldVault.convertToAssets(totalShares) + assets` – so headroom tracks what the vault actually holds.

Team Response

Fixed.

I-05

● INFO

FIXED

The Documented "One Share" Tolerance Is Implemented as One Share-Wei

Description

`_oneShareValue()` is described by its own NatSpec as "One share's asset value, the width of every rounding tolerance in this contract", and three further in-code comments repeat that unit: the deposit path bounds share rounding by "one share's asset value", the withdraw path charges dust "bounded by one share's value", and `_materiallyImpaired` is documented as having "a one-share tolerance".

The implementation computes something else. `convertToAssets()` converts **one share-wei**, not one whole share, so the returned width is `floor(value of 1 share-wei) + 1`. On the live geometry – 18-decimal venue shares over 6-decimal USDG – that is `0 + 1 = 1 wei`, while one *whole* share is worth 1,006,384 wei. The single constant that sets every tolerance in the contract therefore differs from its written specification by roughly six orders of magnitude.

Location of Affected Code

The specification, stated four times:

File: [src/OffyieldVault.sol](https://github.com/shieldify/shieldify/blob/master/src/OffyieldVault.sol)

```

// deposit path
// never more than sent. Share rounding (bounded by one share's asset
// value) stays in the pool instead of being booked as principal, ...

// withdraw path
// genuine rounding dust (bounded by one share's value) to the

// The helper itself
/// @dev One share's asset value, the width of every rounding tolerance in
/// this contract. Rounded up so a tolerance is never zero-width.

// The impairment gate
/// @dev Impairment with a one-share tolerance, so ordinary venue rounding
/// does not read as a loss.

```

The implementation, which converts one share-wei:

File: [src/OffyieldVault.sol#L371-L373](#)

```

function _oneShareValue() internal view returns (uint256) {
    return yieldVault.convertToAssets(1) + 1; // 0 + 1 == 1 wei on the live geometry
}

```

The three consumers that inherit the width:

File: [src/OffyieldVault.sol](#)

```

function deposit(uint256 assets) external nonReentrant whenNotPaused {
    // code
    if (credited + _oneShareValue() < assets) revert UnexpectedTransferAmount();
    // code
}

function withdrawPrincipal(uint256 assets, address receiver) external nonReentrant {
    // code
    if (gap <= _oneShareValue()) {
        // code
    }
}

function _materiallyImpaired(Position storage p) internal view returns (bool) {
    return yieldVault.convertToAssets(p.shares) + _oneShareValue() < p.principal;
}

```

Impact

No loss occurs at the current implementation. The defect is that the contract's own documentation specifies a tolerance six orders of magnitude wider than the one it implements.

Recommendation

Reconcile in the direction of the code, and do not change `_oneShareValue()`. The rounding unit of an ERC-4626 venue is one share-wei, and `convertToAssets(1) + 1` is already the exact tight bound.

Correct the four comments to name the unit they actually describe, e.g. for the helper:

```
/// @dev Asset value of one share-WEI (the venue's ERC-4626 rounding unit),  
/// the width of every rounding tolerance in this contract. Rounded up  
/// so a tolerance is never zero-width. NOT one whole share: widening  
/// this to `convertToAssets[1e18]` would let real venue losses pass as  
/// rounding dust and break `positionValue >= principal`.  
function _oneShareValue() internal view returns (uint256) {
```

Team Response

Fixed.

I-06

● INFO

FIXED

A 2-of-2 Owner Multisig Has No Fault Tolerance

Description

`ops/DEPLOY.md` allows a **2-of-2** multisig for the owner role. With `m == n` there is no redundancy: losing one signer key loses the owner account entirely, and every `onlyOwner` function – `unpause()`, `setGuardian()`, `setCaps()` – becomes permanently uncallable.

The pause asymmetry makes that terminal. `pause()` is callable by the guardian *or* the owner, but `unpause()` is `onlyOwner`. So once one owner key is gone, the next pause can never be lifted. Recovery is not possible either.

User funds are never at risk, since `withdrawPrincipal()` and `exit()` are unpausable. It kills the product, not the money.

Location of Affected Code

File: [ops/DEPLOY.md](#)

```
| **Owner** | Safe multisig (or 2-of-2 timelock if Safe is unavailable on 4863) | ... |
```

Either party can pause, only the owner can lift it:

File: [src/OffyieldVault.sol](#)

```
function pause() external {
  if (msg.sender != guardian && msg.sender != owner()) revert NotGuardianOrOwner();
  _pause();
}

function unpause() external onlyOwner {
  _unpause();
}
```

Impact

One lost signer device permanently disables every admin power: the vault can never be unpaused, the guardian can never be rotated, and ownership can never be migrated. Deposits and `spendFromYield()` stop for good.

Recommendation

Do not use a 2-of-2. Require `m`-of-`n` with `n > m` so one key can be lost without losing the account - **2-of-3** at minimum, **3-of-5** preferred for mainnet.

Also keep every signer key on a separate device and with a separate person: no two signers sharing a laptop, phone, cloud backup, or seed phrase, otherwise, the threshold is cosmetic and one loss takes out the whole set.

Team Response

Fixed.



Your security partner, for the long term

[SHIELDIFY.ORG](https://shieldify.org) · [X.COM/SHIELDIFYSEC](https://x.com/shieldifysec) · [LINKEDIN](#)